applied sciences

MDPI

Article

An Application Programming Interface (API) Sensitive Data Identification Method Based on the Federated Large Language Model

Jianping Wu [1], Lifeng Chen [2,*], Siyuan Fang [3] and Chunming Wu [1,*]

[1] College of Computer Science and Technology, Zhejiang University, Hangzhou 310027, China; wjpself@zju.edu.cn
[2] Shanghai Fushu Technology Co., Ltd., Shanghai 311700, China
[3] Shanghai Fudian Intelligent Technology Co., Ltd., Shanghai 311700, China; fangsiyuan@vconinfo.com
* Correspondence: chenlifeng@hotmail.com (L.C.); wuchunming@zju.edu.cn (C.W.)

Abstract: The traditional methods for identifying sensitive data in APIs mainly encompass rule-based and machine learning-based approaches. However, these methods suffer from inadequacies in terms of security and robustness, exhibit high false positive rates, and struggle to cope with evolving threat landscapes. This paper proposes a method for detecting sensitive data in APIs based on the Federated Large Language Model (FedAPILLM). This method applies the large language model Qwen2.5 and the LoRA instruction tuning technique within the framework of federated learning (FL) to the field of data security. Under the premise of protecting data privacy, a domain-specific corpus and knowledge base are constructed for pre-training and fine-tuning, resulting in a large language model specifically designed for identifying sensitive data in APIs. This paper conducts comparative experiments involving Llama3 8B, Llama3.1 8B, and Qwen2.5 14B. The results demonstrate that Qwen2.5 14B can achieve similar or better performance levels compared to the Llama3.1 8B model with fewer training iterations.

Keywords: federated learning; large language model; sensitive data identification; API; data security

check for updates

Citation: Wu, J.; Chen, L.; Fang, S.; Wu, C. An Application Programming Interface (API) Sensitive Data Identification Method Based on the Federated Large Language Model. *Appl. Sci.* **2024**, *14*, 10162. https://doi.org/10.3390/app142210162

Academic Editors: Grigorios Beligiannis, Georgios A. Tsirogiannis and Douglas O'Shaughnessy

Received: 6 September 2024
Revised: 20 October 2024
Accepted: 31 October 2024
Published: 6 November 2024

Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

APIs, as crucial channels for functional invocations and data flows between applications, have significantly facilitated digital development, with their numbers experiencing exponential growth in recent years. However, due to their openness, APIs have increasingly become a prime target for abuse and hacking attacks, leading to many data breaches. According to the "2023 API Security Status Report" released by Traceable [1], API-related data breaches have significantly increased in recent years. Over the past two years, 60% of surveyed organizations have reported at least one data breach, with 74% of them experiencing three or more incidents. Data from Akamai [2] indicate that API attacks accounted for nearly 30% of web attack types in 2023. Risk analyses of data breaches over the past two years underscore that API security has emerged as the primary risk for data exposure. The access and invocation of APIs involve complex and disorganized transmission and interaction of content data, encompassing diverse data types. Especially with the widespread adoption of AI, API security risks have further escalated. Consequently, the effective management and control of API sensitive data have become a critical research focus and challenge in the field of data security.

Currently, there have been relevant studies on API sensitive data identification. Cheh et al. [3] proposed a semi-automatic method to infer various critical information from API specifications, thereby identifying sensitive and non-sensitive data fields, detecting insecure or high-risk API invocations, and calculating the exposure level of each data field and API call. Faruk et al. [4] reviewed the exposure of application logic and sensitive data

information through APIs, pointing out that such exposure puts patients' data at risk on the internet, thus making them a target for attackers. Sun et al. [5] implemented cross-network communication taint propagation based on dynamic taint analysis and system-level emulation for the first time, enabling the tracking of sensitive data flows in API communications. These studies provide robust support for API sensitive data identification and protection.

API sensitive data identification refers to the process of identifying and protecting data that may contain sensitive information during API interactions to prevent unauthorized access and data breaches. Currently, API sensitive data identification primarily relies on predefined rules and templates, such as regular expressions. While these methods are straightforward, they have limited recognition capabilities when confronted with complex or unknown data formats. Additionally, some research institutions have begun to apply machine learning and deep learning models more extensively, training them to identify sensitive data. For instance, natural language processing (NLP) techniques are used to identify sensitive information within text. Advanced research endeavors have attempted to identify sensitive data through semantic analysis and contextual understanding, which can provide more accurate recognition but also requires higher computational resources and more complex algorithms.

With the rapid development of ChatGPT and similar generative AI technologies, generative AI security use cases based on large language models (LLMs) have emerged as the mainstream trend in network security intelligence. LLMs have demonstrated immense potential in security verticals such as security classification [6] and content moderation [7], positioning them to address various cyber-threats more effectively and lay a solid foundation for a safer and healthier online environment. ToolLLM [8] developed the ToolBench dataset and DFSDT algorithm, enabling LLMs to successfully handle complex tasks involving numerous real-world APIs. Simultaneously, LLMs themselves pose significant security concerns, including the safety of their content output, interpretability, potential for the leakage of users' private data, and vulnerability to theft. Insecure LLMs can inflict immeasurable losses on enterprises, such as breaches of privacy and sensitive information.

This paper proposes effectively addressing the robustness and accuracy issues of API sensitive data identification based on rule-based libraries and conventional machine learning by leveraging federated learning and large language model technologies. Additionally, it tackles the security issue of sensitive data leakage when applying general-purpose large language models through federated learning. The innovative aspects of this paper are as follows:

- Enhanced accuracy and generalization for API sensitive data identification: Compared to existing open-source large language models, the dedicated large language model for API sensitive data identification, APILLM, possesses powerful contextual understanding and feature extraction capabilities, significantly improving the accuracy and generalization of API sensitive data identification.
- A solution is provided for the issue of computational resource depletion in the integrated application of federated learning and large models. A parameter-efficient LoRA fine-tuning method is proposed, and a continual learning approach is introduced to prevent crucial knowledge in the global model from being forgotten during local model training, thereby solving the issue of computational resource exhaustion in the combined application of federated learning and large models.
- A solution is provided for the issue of API privacy data protection. Through federated learning, this paper effectively addresses the issue of potentially exposing user privacy from each data source and compromising information security when multiple data sources jointly train a large language model (LLM).

2. Related Works

2.1. API Sensitive Data Identification

The primary aim of API attacks is to steal and tamper with sensitive data, making the integration of API protection measures with sensitive data monitoring a practical and effective approach. API sensitive data identification serves as a crucial prerequisite

for safeguarding data privacy and security. Intelligent algorithms automate the process of identifying and classifying sensitive information from vast amounts of data, thereby reducing human errors and enhancing efficiency. Currently, two primary categories of algorithms and technologies are employed for sensitive data identification: rule-based and machine learning-based.

1. Rule-Based Algorithms: Rule-based methods utilize predefined patterns and rules to match sensitive data. For instance, regular expressions can be employed to detect common sensitive information such as social security numbers, credit card numbers, email addresses, etc. These patterns can be manually defined or automatically generated. The advantages lie in their simplicity, directness, and ease of implementation, as well as their strong interpretability with clear and concise rules. However, they suffer from difficulties in maintenance and updating, and lack flexibility in dealing with unknown patterns and highly variable data.
2. Machine Learning Algorithms: Machine learning algorithms identify sensitive data by training models. These models can automatically recognize and classify sensitive information within data. In supervised learning, classifiers are trained using labeled training data, with common algorithms including Support Vector Machines (SVMs), Random Forest, and K-Nearest Neighbor (KNN), primarily used for identifying sensitive information in structured data. In unsupervised learning, sensitive data are identified by discovering patterns and anomalies within the data, with algorithms such as clustering (e.g., K-Means) and Isolation Forest commonly employed for detecting anomalies and sensitive information in unlabeled data.
3. Hybrid Methods: Hybrid methods combine the strengths of multiple technologies and algorithms to improve the accuracy and efficiency of sensitive data identification. For example, a hybrid approach might integrate rule-based methods with machine learning methods, initially screening data with rules and then refining the classification with machine learning models.
4. Privacy-Enhancing Technologies: Privacy-enhancing technologies enable sensitive data identification while ensuring data privacy. For instance, federated learning allows devices to collaboratively train models without sharing raw data, thereby protecting data privacy. This paper focuses on implementing API sensitive data identification through the fusion of federated learning and large language models.

Sensitive data identification is often combined with privacy-preserving computing technologies such as federated learning, differential privacy, and homomorphic encryption, to enable data analysis and processing while protecting user privacy. In the field of smart healthcare, applications of sensitive data identification based on federated learning have addressed privacy protection issues in electronic health record management, medical image analysis, and other areas, such as predicting heart disease hospitalizations based on patients' electronic health records [9]. A collaborative model based on federated learning and blockchain has been constructed using data from multiple institutions to identify COVID-19 patients from lung CT images [10]. In the field of smart transportation, [11] proposes a traffic resource scheduling method based on federated learning, which significantly improves group fairness and travel efficiency.

Most of the aforementioned research efforts focus on designing context understanding capabilities from the perspectives of machine learning and deep learning, resulting in poor generalizability and high false positive rates, and thus failing to address complex and diverse application scenarios. Therefore, leveraging both deep learning and pre-trained large models to design a large model for government security and solving related challenges is another research focus of this paper.

2.2. Federated Learning

Federated learning is a distributed machine learning framework introduced by McMahan et al. [12] from Google in 2017 and implemented in language prediction models on smartphones. It enables the training of a unified machine learning model using datasets distributed across

different mobile devices without compromising the privacy of individual user data. Each FL client performs local training, and the computed updates are transmitted to a central entity known as the aggregator. This aggregator is responsible for generating a refined global model by processing the received updates using an aggregation function. As shown in Figure 1.

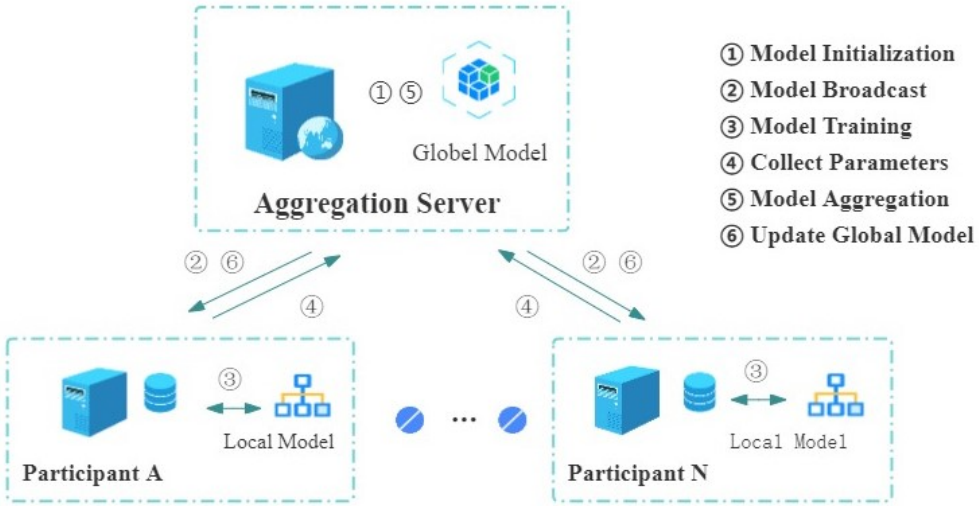


Figure 1. Federated learning model training process.

Federated learning has found extensive application in large language models. Reference [13] proposes a federated learning framework named FedJudge, which is designed for the efficient and effective fine-tuning of legal large language models. This framework leverages parameter-efficient fine-tuning methods, updating only a small number of additional parameters during federated learning training. In [14], a federated logic rule learning method called FedLogic is introduced, which presents theoretical formalization and interactive simulation of the multi-domain Chain-of-Thought (CoT) prompt selection dilemma in the context of federated large models. Meanwhile, ref. [15] presents low-parameter federated learning (LP-FL), which combines rapid learning from a few queries with large language models (LLMs) and efficient communication and federation techniques.

2.3. Large Language Models

Large language models are powerful text processing frameworks developed based on deep learning technology. In November 2022, OpenAI released ChatGPT (Chat Generative Pre-trained Transformer), a conversational large language model [16]. This model allows users to interact through natural language dialog, enabling various natural language understanding and natural language generation tasks, including automatic question answering, text classification, automatic summarization, machine translation, and chat conversations. ChatGPT is fine-tuned from GPT-3.5, a Generative Pre-trained Transformer (GPT) model. Shortly after the release of ChatGPT, OpenAI subsequently unveiled the GPT-4 model. Its context window length may have been increased from 4096 tokens in GPT-3.5-turbo to 32,768 tokens. Additionally, GPT-4 is a multimodal model capable of recognizing and extracting image information and providing textual feedback.

LLMs can be classified into two types: open-source and closed-source models. Open-source LLMs, such as Llama [17] and Mixtral [18], offer researchers transparency and the ability to customize and fine-tune models for specific cybersecurity tasks. This adaptability is particularly valuable in cybersecurity scenarios involving private data and customization requirements. However, open-source LLMs may lack the performance and scale of closed-source LLMs. On the other hand, closed-source LLMs (often referred to as commercial LLMs, like ChatGPT [16] and Gemini [19]) provide state-of-the-art performance, are maintained by commercial entities, and typically have restricted access. The relevant analysis of ChatGPT 4, LLaMA 3.1 8B, and Qwen 2.5 14B is shown in Table 1.

Table 1. Comparison of Performance Analysis of ChatGPT 4 , LLaMA3.1 8B, and Qwen2.5 14B.

Feature/Model	ChatGPT 4	LLaMA3.1 8B	Qwen2.5 14B
Model Size	9 billion parameters	8 billion parameters	14 billion parameters
Training Data	Bilingual focus	Multi-domain datasets	Large-scale multi-domain datasets
Computational Power	High	High	Very high
Energy Consumption	High	High	Very high
Application Scenarios	Chinese NLP, bilingual tasks	Various NLP tasks	Wide-range NLP tasks
Language Understanding	Extremely strong	High	Very strong
Generative Ability	Extremely strong	High	Extremely strong
Conversational Ability	Very strong	High	Very strong
Robustness	Extremely high	High	Very high
Training Time	Very long	Long	Very long
Scalability	Good	Good	Moderate to good

The cybersecurity landscape faces escalating threats, necessitating intelligent and efficient solutions to counter complex and evolving attacks [20–22]. LLMs present new opportunities for the cybersecurity community [23,24]. Trained on vast amounts of data, LLMs acquire extensive knowledge and develop robust comprehension and reasoning abilities, providing a powerful decision-making tool for cybersecurity.

3. Method Design

3.1. Method Overview

LLMs, leveraging the Transformer architecture and self-supervised pre-training strategies, have demonstrated remarkable capabilities in understanding and content generation [15,17]. However, developing a specialized LLM for sensitive data identification from scratch requires significant computational resources, which is impractical for most research teams. Therefore, it is necessary to select a general-purpose LLM as the base model and fine-tune it accordingly. Currently, it is challenging to determine which large language model performs better for specific tasks. From the perspective of data security, this paper chooses the newly released Llama3 (8B) as the experimental model, which is fully open-source and among the most advanced large language models currently available. The proposed framework in this paper includes the Llama3 base model, pre-training, LoRA fine-tuning, and other components. During the pre-training phase, names, identity card numbers, phone numbers, home addresses, etc., are labeled as sensitive data. The process of sensitive data identification based on the LLM's API is illustrated in Figure 2.

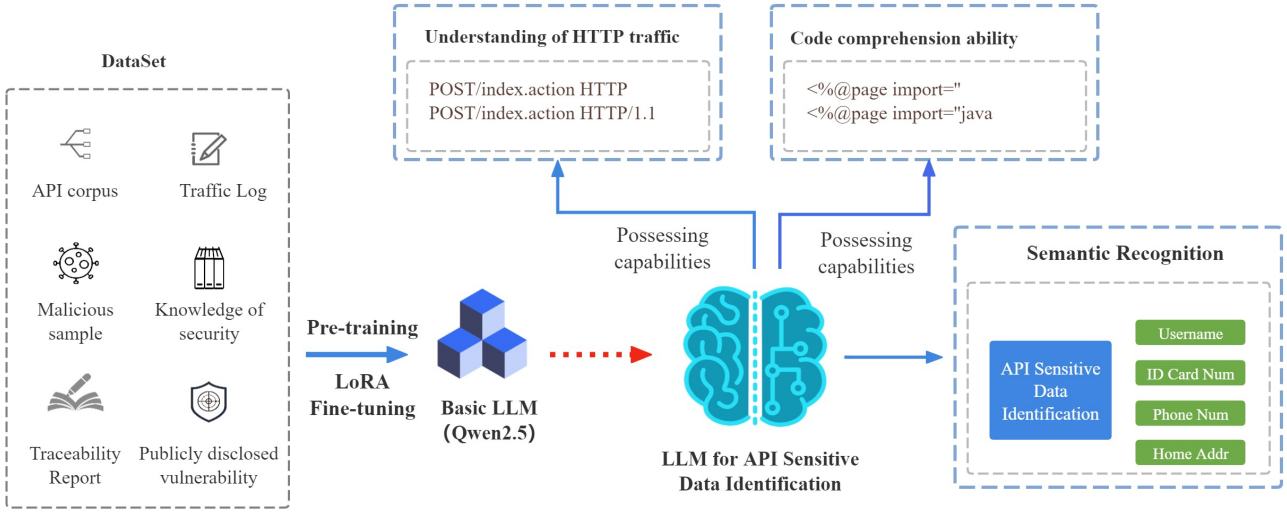


Figure 2. Sensitive data identification process of APILLM.

3.2. Network Architecture

The primary scenarios for data flow based on APIs involve interactions between application servers and between application servers and application databases, as depicted in Figure 3. Each application server is equipped with an LLM for the real-time detection of sensitive data within APIs, enabling early warning and the automatic suspension of risky APIs. The federated learning server conducts protocol analysis and semantic recognition across full traffic flows, incorporating various networks into fine-grained monitoring channels. This approach eliminates potential security risks associated with the LLM itself during the detection of illegal leaks of sensitive API data, effectively addressing data security risks in specific scenarios. Additionally, by displaying the fine-grained flow path of sensitive API data and conducting correlation analysis based on users and paths, the system enables precise tracing and accountability in the event of data breaches.

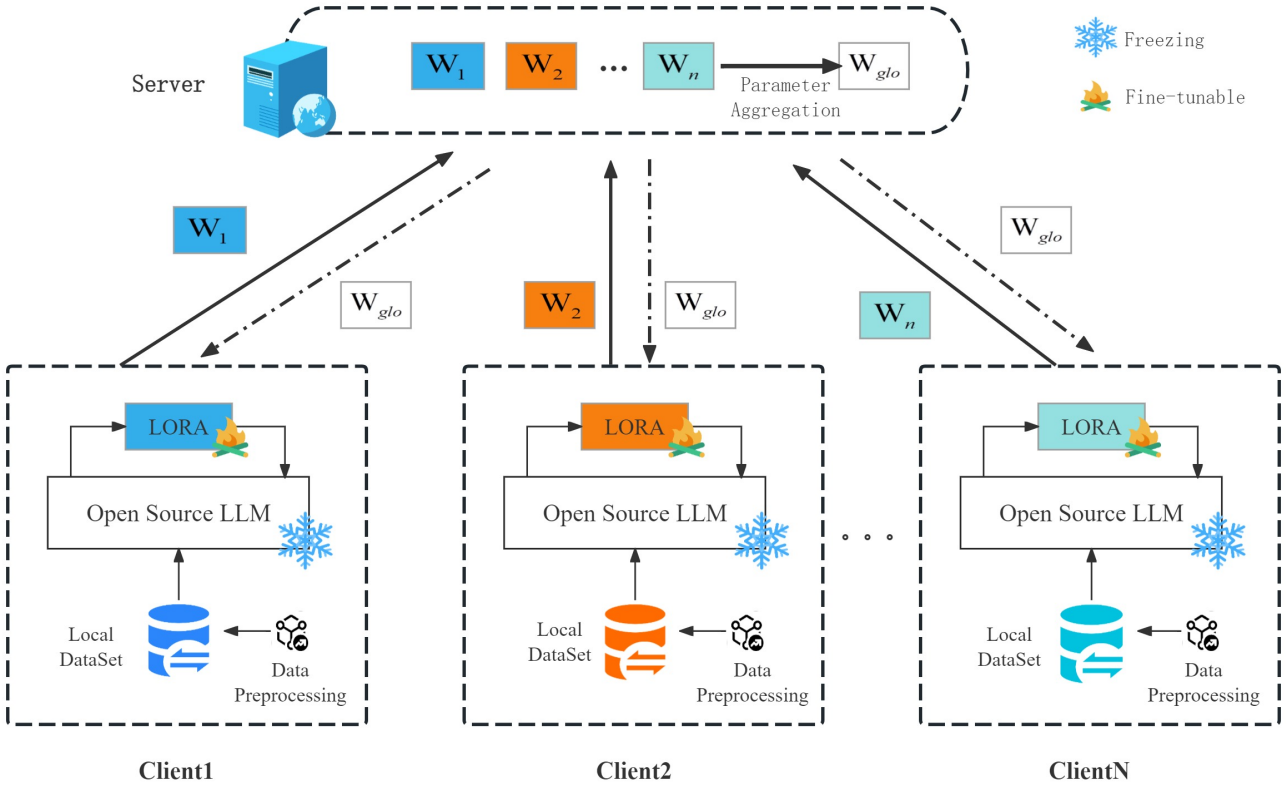


Figure 3. The training process of LoRA based on federated large models.

3.3. Data Preprocessing

To evaluate the effectiveness of the proposed method, this paper selects the publicly available Web API Dataset [25] from Github for experimentation. This dataset contains information on various Web APIs, including their descriptions, parameters, requests, and responses, totaling 1123 entries. Among them, there are 624 sensitive data entries and 499 non-sensitive data entries. This dataset can be used to train models to understand API functionality, input/output types, semantic relationships, and more. The dataset sources include public API documentation and online API platforms.The raw dataset may suffer from issues such as missing values, outliers, noise, or data imbalance. Data preprocessing steps are essential to enhance sample quality. Given the specificity of API sensitive data samples, the main preprocessing techniques applied include removing duplicate samples, eliminating empty samples, URL decoding, and removing redundant white spaces. The experimental data are cleaned and formatted, and manual annotation is performed to

indicate whether each data point contains sensitive information, aligning with the model's input requirements.

3.4. Instruction Tuning

LLMs possess a vast number of parameters, and fully fine-tuning an LLM is extremely resource-intensive, leading to significant computational costs that limit feasibility for clients with limited computational capabilities. Furthermore, using traditional FL algorithms (such as FedAvg) to aggregate and distribute LLM parameters increases the communication overhead of the FL system. The standard FL workflow is not applicable in this context, and both factors reduce the efficiency of LLM fine-tuning. Fine-tuning is a transfer learning technique that involves additional training on a pre-trained model to adapt it to a specific task or dataset. In this paper's experiments, the Low-Rank Adaptation (LoRA) fine-tuning method is adopted to optimize the Llama3 large language model. LoRA is one of the most effective parameter-efficient fine-tuning paradigms, offering notable advantages in cross-task generalization and privacy protection. QLoRA [26] quantizes the pre-trained model to 4 bits, significantly reducing memory usage while maintaining performance comparable to that of 16-bit fine-tuning. GLoRA [27] can dynamically adjust low-rank updates, enabling better model control and adaptability. AdaLoRA [28], on the other hand, automatically allocates the budget for fine-tunable parameters based on their importance, enhancing the model's performance across different tasks and datasets.

The fundamental principle of LoRA is to freeze the pre-trained model parameters and introduce trainable bypass matrices (low-rank separable matrices) at each layer of the Transformer. These bypass outputs are then added to the initial path outputs before being fed into the network, and only the parameters of these newly added bypass matrices are trained. The low-rank separable matrices consist of two matrices: the first matrix reduces the dimensionality, and the second matrix increases it, with an intermediate dimension of r, simulating the intrinsic rank. These two low-rank matrices significantly reduce the number of parameters [29].

Given a dense neural network layer parameterized by $W_0 \in {}^{d\times k}$, to adapt it to a specific task, $\Delta W \in {}^{d\times k}$ is updated to obtain a new layer parameterized by $W = W_0 + \Delta W$. In the context of LoRA fine-tuning, $r \ll \min(d, k)$, where $\Delta W = BA$, with $A, B \in {}^{d\times k}$. During the training process, W_0 is fixed and no longer updated via gradient descent; only A and B are trained. For an input x, the forward propagation of the model is updated as follows:

$$h = W_o x + \Delta W x = W_o x + BAx \tag{1}$$

When adapting LLMs to vertical domains such as healthcare, finance, tourism, and others, the available training data may be privately owned by multiple clients. In such scenarios, to achieve decentralized training data while maintaining data localization and fine-tuning LLMs, federated learning methods are commonly employed. In federated learning, clients typically compute weight updates locally and then share these updates with other clients to globally update the LLM. However, this approach can impose communication and computational costs on clients. Nevertheless, LoRA, with its parameter efficiency and pluggability, can reduce communication costs and lower the demand for computational resources [30]. LoRA enhances the overall efficiency and scalability of federated learning. The process of LoRA fine-tuning based on federated learning is illustrated in Figure 3.

Step 1: On the server, the pre-trained large language model LLM is initialized as a global model, most of the parameters in the LLM's weight matrix are frozen, and the global model parameters w_{glo} are issued.

Step 2: Each client uses the global LLM as the initial local LLM and introduces LoRA to achieve personalized local knowledge learning. During the fine-tuning process of LoRA, two low-rank matrices, A and B, are introduced to adjust the weight matrix of the model.

$$W_{new} = W_0 + A \cdot B \tag{2}$$

where W_0 is the original model weight, and A and B are the learned low-rank matrices used for fine-tuning.

Step 3: The client performs training based on the local LLM, calculates the anomaly detection regularization term and loss function related to the global LLM, evaluates the usefulness and safety of the response, and performs backpropagation to adjust the parameters of the local model, especially updating the parameters of the LoRA part, as shown in Algorithm 1. During the backpropagation process, the gradient of the loss function concerning the new weights is calculated as follows:

$$\nabla WL = \nabla W_{new} L \cdot (I + \frac{\partial (A \cdot B)}{\partial W}) \tag{3}$$

where $\nabla W_{new} L$ is the gradient of the loss function L concerning the new weights, I is the identity matrix, and $\partial(A \cdot B)/\partial W$ is the derivative of the product of low-rank matrices concerning the original weights.

Algorithm 1 Federated Learning-based LoRA Client Training Process

Input: *Global model parameters* w_{glo}, *Local Dataset* D_i
Output: *Local model update* $\triangle w_i$

While Server returns update **do**
    Receive global model parameters from Server w_{glo}.
    For number of iterations k = 1, . . . , K do:
        Train model using local data D_i.
        Train local model update $\triangle w_i$ using LORA.
    end for
    $Enc(pk, w_i) \rightarrow \triangle w_i$.
    //Encrypt the local updates w_i, where pk is the encryption operator.
    Send encrypted updates $\triangle w_i$ to Server.
End While

Step 4: The server aggregates the sparse adjustable layers (i.e., the unfrozen parts of the LLM) from all clients to obtain an updated global LLM, and sends the unfrozen parts of the global LLM to all clients, as shown in Algorithm 2.

Algorithm 2 The parameter aggregation process on the LORA server based on FL

Input: $Pre-trained\ model\ parameters\ w_{pro}, number\ of\ clients\ N$
Output: *Global model parameters* w_{glo}

$w_{glo} \leftarrow w_{pre}$
While not converged **do**
    For each client i in 1 to N do
        Send the global model parameters w_{glo} to client C_i
    End For
    For each client i in 1 to N do
        Receive the encrypted update $\triangle w_i$ from client C_i
        if anomalydetect ($\triangle w_i$) is normal then
            $Dec(sk, \triangle w_i) \rightarrow w_i$.
            $w_{glo} = (w_1 + w_2 + \cdots + w_n)/N$
        end if
    end for
    CheckConvergence(w_{glo}) .
End While

The global loss function combines the loss from each client and the regularization term for anomaly detection:

$$L_{globel}(w_{glo}) = \sum_{i=1}^{N} w_i L_i(w_{glo}; D_i) + \lambda \sum_{i=1}^{N} R(\triangle w_i) \quad (4)$$

where $L_i(w_{glo}; D_i)$ is the loss function of client C_i, and $R(\triangle w_i)$ is the regularization term for anomaly detection. Secure aggregation is shown in Algorithm 3.

Algorithm 3 Security Aggregation Process for LoRA Servers Based on FL

Input: $\{\triangle w_1, \triangle w_2, \cdots, \triangle w_n\}$
Output: *Global model parameters* w_{glo}

$w_{sum} \leftarrow 0$
$\triangle w_{\text{glo}} \leftarrow 0$
    For client C_i to N do
        If TrustScore(i) > threshold then
            $\triangle w_{\text{glo}} \leftarrow \triangle w_{\text{glo}} + \triangle w_i$.
            $w_{sum} \leftarrow w_{sum} + 1$
        end if
    End For
    If $w_{sum} > 0$ then
        $\triangle w_{\text{glo}} \leftarrow \triangle w_{\text{glo}} / w_{sum}$
    End if
return w_{glo}

The global loss function plays a crucial guiding role in both federated learning and the LoRA architecture, determining the direction of model parameter updates. Both the server side and the client side aim to minimize the global loss function for model optimization.

Step 5: Steps 3 and 4 are repeated until the convergence criteria for federated learning are met.

4. Experiments and Analysis

4.1. Experimental Environment

The experiments were conducted in a high-performance computing environment equipped with an NVIDIA GeForce RTX 4090 GPU (NVIDIA Corporation, Santa Clara, CA, USA). The RTX 4090, NVIDIA's high-end graphics processing unit, is renowned for its formidable computational power and efficient parallel processing capabilities, making it ideally suited for executing deep learning and other data-intensive tasks. The hardware platform utilized in the experiments provided the necessary computational resources to support the fine-tuning and training of large-scale language models. The operating system was Ubuntu 20.04, and Python 3.11.9 served as the development framework. To accelerate the inference of large language models, open-source tools such as llama.cpp were employed for quantized inference, and unsloth was applied for instruction tuning.

Model Selection: Based on the minimal viable analysis in academic research, this experiment selects five models for experimental analysis to observe the impact of model quantization and model versions on the experimental results. The selected models are Llama3 8B, Llama3 8B 4bit, Llama3.1 8B Instruct, Llama3.1 8B Instruct 4bit, and Tongyi Qianwen Qwen2.5 14B. These five versions of models have low hardware requirements, can be deployed locally, and all support 128K contexts, meeting the vast majority of needs.

To evaluate the effectiveness of sensitive data identification, this paper employs the confusion matrix as an assessment metric, where true positives (TPs) represent the number of instances correctly predicted by the model as belonging to the positive class; true negatives (TNs) represent the number of instances correctly predicted by the model as belonging to the negative class; false positives (FPs) represent the number of negative class instances incorrectly predicted by the model as belonging to the positive class; and false

negatives (FNs) represent the number of positive class instances incorrectly predicted by the model as belonging to the negative class. Using these four values, we can calculate the accuracy, precision, recall, and F1-score.

Accuracy: The ratio of the number of correctly predicted samples to the total number of samples. Higher accuracy indicates higher prediction accuracy of the model, as shown in Equation (5).

$$Accuracy = \frac{TP + TN}{N} \tag{5}$$

Precision: The proportion of all predicted positive instances that are actually positive, as shown in Equation (6).

$$Precision = \frac{TP}{TP + FP} \tag{6}$$

Recall: This is a metric that measures the proportion of actual positive instances that are correctly predicted by the classification model, as shown in Equation (7).

$$Recall = \frac{TP}{TP + FN} \tag{7}$$

F1-Score: The harmonic mean of precision and recall, as shown in Equation (8).

$$F1 - Score = \frac{2 \times Precision \times Recall}{Precision + Recall} \tag{8}$$

4.2. Experiment on the Impact of Training Epochs on Model Performance

After each training epoch, the loss of the model is recorded, and true positives (TP) and true negatives (TN) are recorded every 300 training epochs for the subsequent calculation of performance metrics. The relationship between the number of training epochs and the loss function value is illustrated in Figure 4.

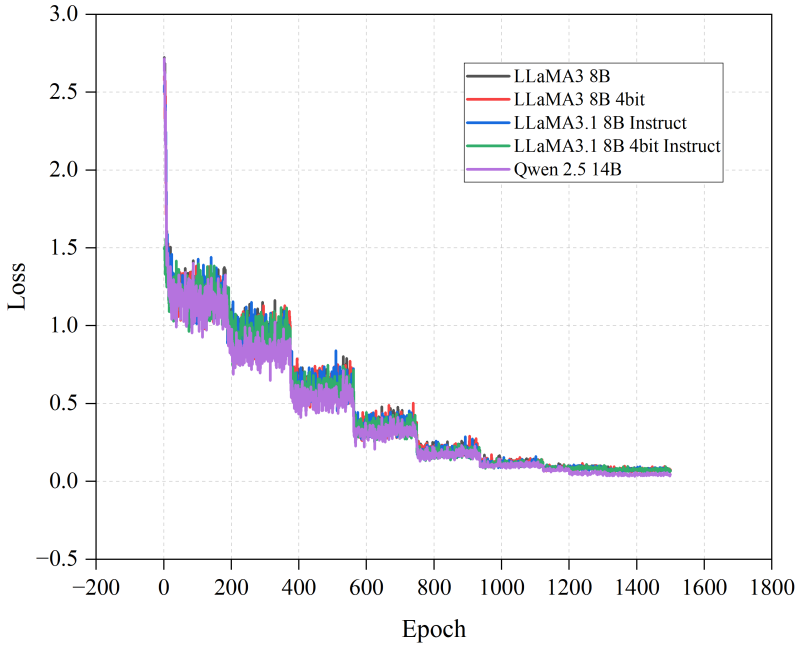


Figure 4. The impact of fine-tuning the number of training epochs on the loss function.

In this paper, the LLaMA3 model, with versions including LLaMA3 8B, LLaMA3 8B 4bit, LLaMA3.1 8B Instruct, LLaMA3.1 8B Instruct 4bit, and Qwen2.5 14B, was fine-tuned on the training set for 1500 epochs, taking 4.5 h, 2.6 h, 4.8 h, 2.8 h, and 6.5 h respectively. For large language models with massive parameters, this time overhead is considered acceptable [31]. During the fine-tuning phase, the loss function of the model after each iteration was tracked and recorded, and a nonlinear function was used for fitting. As shown in the experimental results, the loss function value rapidly decreased with the increase in the number of iterations. After exceeding 1120 iterations, the loss function value of the model remained stable, indicating that the model has good convergence.

As can be seen from Figure 4, the Qwen2.5 14B model exhibits the fastest decrease in the loss function. The LLaMA3 8B and LLaMA3.1 8B Instruct versions outperform their 4-bit quantized counterparts in terms of the speed of the loss function reduction. This may be due to the fact that quantization may introduce some information loss, affecting the model's learning efficiency.

The experimental results show that as the number of training epochs increases, the loss function value of the model decreases significantly, indicating that the model gradually adapts to the training data during the learning process. In the early stages, the model's performance improves rapidly, which may be because the model quickly adjusts from its pre-trained state to the specific task. As the number of iterations increases, the rate of performance improvement slows down until a stable state is reached, suggesting that the model has fully learned the patterns in the training data.

During the fine-tuning process, the model is evaluated every 300 epochs. The evaluation criteria encompass five metrics in total: the discriminative ability of sensitive data on the validation set, precision, recall, F1-score, and false positive rate. An iteration epoch vs. model performance evaluation curve is illustrated in the Figure 5 below.

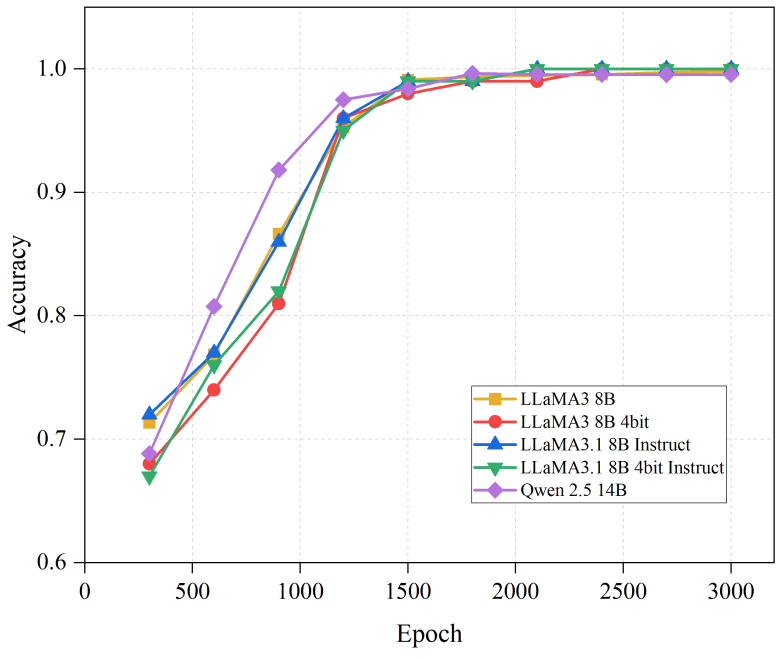


Figure 5. The impact of the number of iterations on model accuracy.

The model's accuracy is approximately 70%. As the number of iteration epochs increases, this ability improves significantly. Overall, the LLaMA3 8B and LLaMA3.1 8B Instruct versions perform better than the LLaMA3 8B 4bit and LLaMA3.1 8B Instruct 4bit versions in terms of model performance. After completing 2100 epochs of fine-tuning, this capability metric approaches 100%. The convergence speed of the accuracy of the Tongyi Qianwen Qwen2.5 14B model is faster as the number of iterations increases, reaching close to 100% after completing 1800 iterations.

The API sensitive data recognition ability of the original model without fine-tuning is only 65%, which is close to the baseline recognition rate of 50% using a random discrimination method, indicating that the model does not exhibit effective sensitive data recognition capabilities in its initial state. After 2100 epochs of fine-tuning, its recognition ability approaches 100%. The Tongyi Qianwen Qwen2.5 14B model also demonstrates a relatively fast convergence speed. As shown in Figure 6.

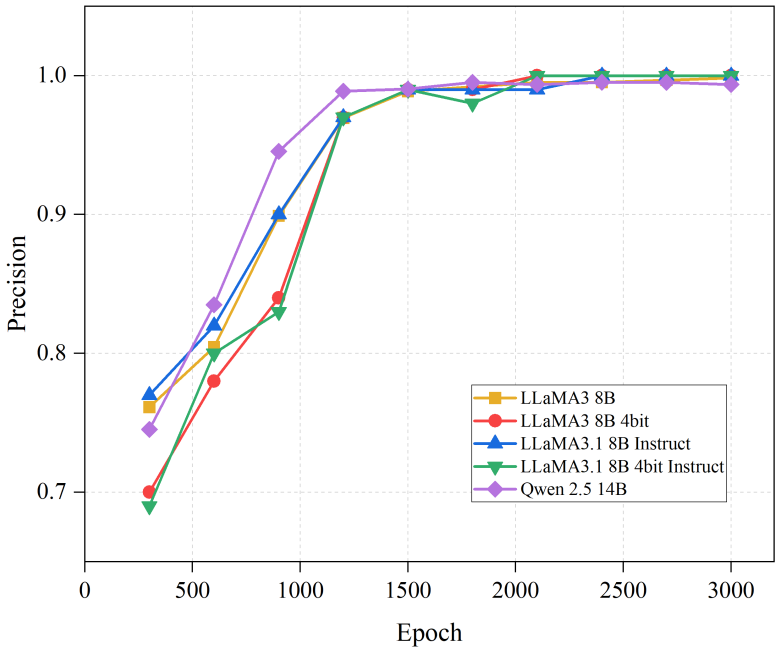


Figure 6. The impact of the number of iterations on model precision.

Figures 7 and 8 demonstrate the changes in recall rate and F1-score as the number of iterations increases. From the figures, it can be observed that both the recall rate and F1-score rapidly improve with an increase in the number of training iterations. When the number of fine-tuning iterations reaches 1500, the recall rate and F1-score approach 100%, further validating the enhancement in model performance.

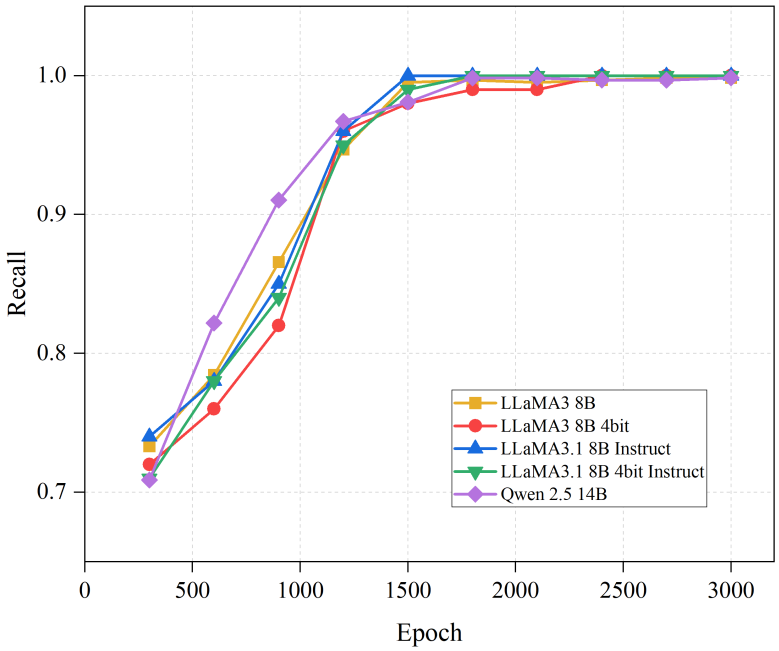


Figure 7. The impact of the number of iterations on model recall.

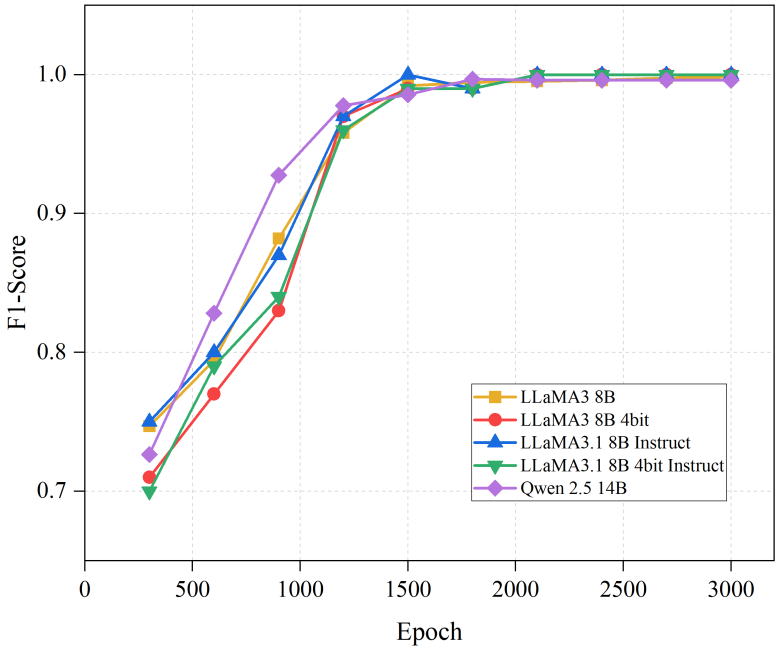


Figure 8. The impact of the number of iterations on model F1$_s$*core*.

As shown in Figure 9. The prototype model's false alarm rate was approximately 30%, but as its capabilities improved, the false alarm rate decreased significantly. After 2500 iterations of fine-tuning, the false alarm rate was reduced to 0. The experimental results indicate that more iterations can lead to better model performance. To balance computational cost and model performance, it is recommended to perform fine-tuning for more than 2500 iterations.

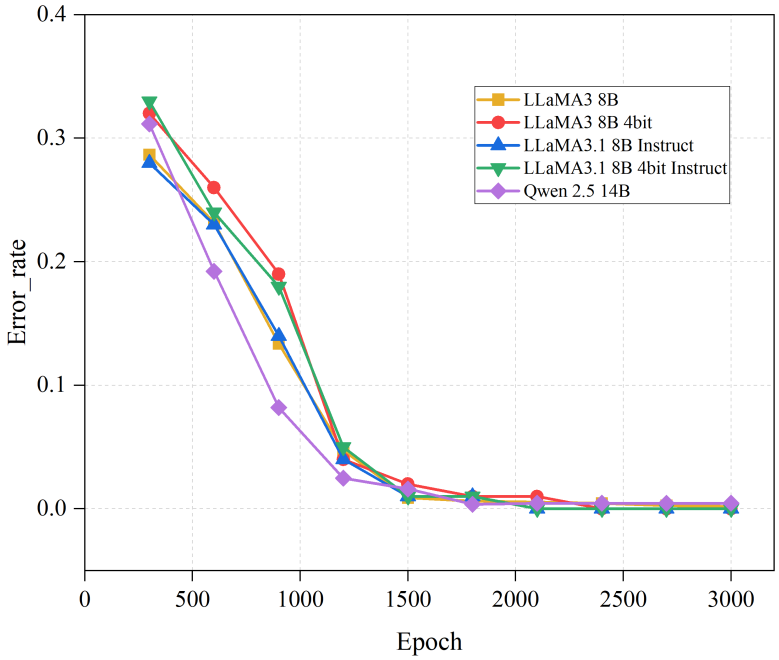


Figure 9. The impact of the number of iterations on model error rate.

5. Discussion

To further explore the impact of fine-tuning on model performance, our experiment analyzes the influence of the number of fine-tuning samples on model performance. Starting from 0 or 1, the number of fine-tuning samples is doubled in each iteration until all

samples are used. After each fine-tuning iteration, TPs (true positives) and TNs (true negatives) are recorded for the subsequent calculation of performance metrics. A curve graph depicting the model's performance evaluation based on the number of samples includes four indicators: accuracy, precision, recall, and F1-score. An analysis of these four metrics reveals that after fine-tuning with 1100 samples, the model's performance stabilizes and reaches its optimal state, as shown in Figure 10.

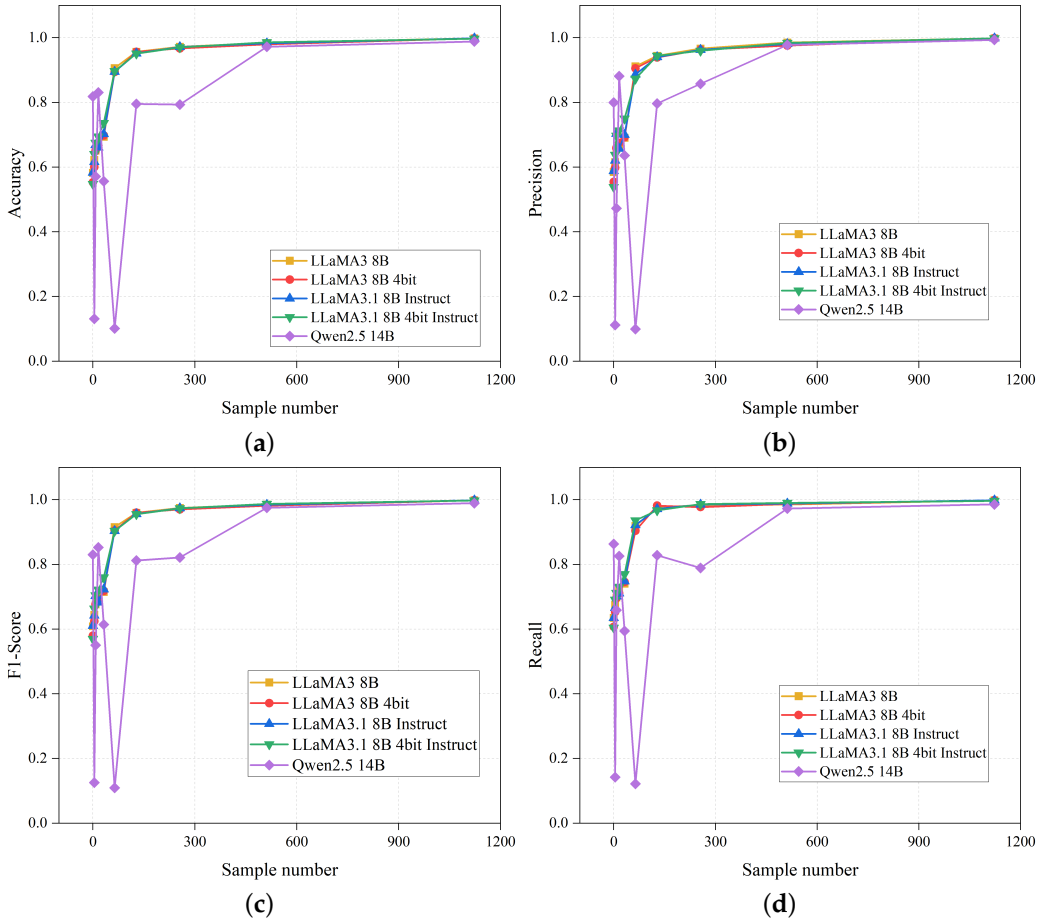


Figure 10. Analysis of the impact of fine-tuning sample size. (**a**) The impact of th sample number on accuracy. (**b**) The impact of the sample number on precision. (**c**) The impact of the sample number on F1-Score. (**d**) The impact of the sample number on Recall.

In the experiments on the number of fine-tuning samples, it was observed that model performance improved as the number of samples increased. This suggests that more training data can help the model generalize better, thereby enhancing its performance on the validation set. However, once the number of samples reaches a certain threshold, the performance improvement stabilizes. This may be because the model has become sufficiently complex to capture all relevant patterns in the data, or due to the emergence of overfitting.

In assessing model performance, this paper pays special attention to metrics such as accuracy, precision, recall, F1-score, and false positive rate. An increase in accuracy indicates an enhanced ability of the model to correctly identify sensitive data. An improvement in recall suggests that the model performs better at not missing sensitive data. The F1-score, as the harmonic mean of accuracy and recall, provides a comprehensive measure of performance. A reduction in the false positive rate signifies progress in reducing the misidentification of non-sensitive data by the model.

Through the experimental performance analysis in this paper, it can be concluded that the proposed federated learning approach combined with large models such as Llama3,

Llama3.1, Qwen 2.5, and the LoRA instruction fine-tuning technique is effective in improving model performance. The model demonstrates good performance improvements with increasing training epochs and fine-tuning sample sizes. Meanwhile, a comparative analysis of the performance of the five large models selected in this chapter reveals that the Qwen2.5 14B model can achieve convergence with fewer iterations. This means that under the same training conditions, the Qwen2.5 14B model can approximate the optimal solution faster, achieving similar or better performance levels compared to the Llama3.1 model with fewer training iterations. This rapid convergence characteristic makes the Qwen2.5 14B model more efficient in practical applications, saving significant training time and computational resources. Especially in scenarios requiring rapid response and iteration, this advantage of the Qwen2.5 14B model is particularly prominent. Furthermore, the model's performance on key performance indicators also demonstrates its potential in practical applications. Future work will focus on further optimizing model performance and exploring more efficient federated learning strategies.

6. Conclusions

The field of API sensitive data identification is rapidly evolving, transitioning from traditional rule-based matching methods to smarter, automated, and highly compliant solutions. This trend will empower businesses with stronger data protection capabilities while also addressing evolving security threats and regulatory requirements. Through experimental data and methodologies, we can evaluate the API sensitive data identification methods based on federated learning and large models in terms of enhancing the accuracy and efficiency of sensitive data classification and recognition, all while preserving data privacy. The data suggest that this approach excels in collaborative training across multiple data sources, demonstrating significant application value and potential for wider adoption. In the future, the development directions for addressing API sensitive data leakage based on federated large models will involve multiple aspects, such as technological innovation and integration, as well as enhanced compliance and regulation. These development directions will contribute to the construction of a safer, more efficient, and flexible AI ecosystem, ensuring user privacy and data security.

Author Contributions: Methodology, J.W., L.C., S.F. and C.W.; formal analysis, S.F. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the "Pioneer" and "Leading Goose" R&D Programs of Zhejiang (2024C01066).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original contributions presented in the study are included in the article, further inquiries can be directed to the corresponding authors.

Conflicts of Interest: Author Lifeng Chen was employed by the company Shanghai Fushu Technology Co., Ltd., and author Siyuan Fang was employed by the company Shanghai Fudian Intelligent Technology Co., Ltd. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

References

1. 2023 API Security Status Report. Available online: https://www.traceable.ai/2023-state-of-api-security (accessed on 5 August 2024).
2. Salt Labs. API Predictions for 2024. Available online: https://salt.security/blog/api-predictions-for-2024 (accessed on 5 August 2024).
3. Cheh, C.; Chen, B.B. Analyzing OpenAPI Specifications for Security Design Issues. In Proceedings of the 2021 IEEE Secure Development Conference (SecDev), Atlanta, GA, USA, 18–20 October 2021; Volume 10, pp. 15–22.
4. Faruk, M.J.H.; Patinga, A.J.; Migiro, L.; Shahriar, H.; Sneha, S. Leveraging Healthcare API to transform Interoperability: API Security and Privacy. In Proceedings of the 2022 IEEE 46th Annual Computers, Software, and Applicationis Conference (COMPSAC 2022), Los Alamitos, CA, USA, 27 June–1 July 2022; pp. 444–445.
5. Sun, R.H.; Wang, Q.X.; Guo, L. Research Towards Key Issues of API Security. *Cyber Secur.* **2022**, *1506*, 179–192.

6. Vörös, T.; Bergeron, S.P.; Berlin, K. Web content filtering through knowledge distillation of large language models. In Proceedings of the 22nd IEEE/WIC International Conference on Web Intelli-gence and Intelligent Agent Technology (WI-IAT), Venice, Italy, 26–29 October 2023; pp. 357–361.
7. Kumar, D.; Abuhashem, Y.A.; Durumeric, Z. Watch your language: Large language models and content moderation. In Proceedings of the Eighteenth International AAAI Conference on Web and Social Media, Buffalo, NY, USA, 3–6 June 2024; AAAI Press: Washington, DC, USA, 2024; Volume 18, pp. 865–878.
8. Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; Qian, B.; et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv* **2023**, arXiv:2307.16789.
9. Brisimi, T.S.; Chen, R.; Mela, T.; Olshevsky, A.; Paschalidis, I.C.; Shi, W. Federated learning of predictive models from federated Elec-tronic Health Records. *Int. J. Med. Inform.* **2018**, *112*, 59–67. [CrossRef] [PubMed]
10. Kumar, R.; Khan, A.A.; Kumar, J.; Golilarz, N.A.; Zhang, S.; Ting, Y.; Zheng, C.; Wang, W. Blockchain-federated-learning and deep learning models for COVID-19 detection using CT Imaging. *IEEE Sens. J.* **2021**, *21*, 16301–16314. [CrossRef] [PubMed]
11. Shi, D.; Tong, Y.; Zhou, Z.; Song, B.; Lv, W.; Yang, Q. Learning to assign: Towards fair task assignment in large-scale ride hailing. In Proceedings of the 27th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), Singapore, 14–18 August 2021.
12. Mcmahan, H.B.; Moore, E.; Ramage, D.; Hampson, S.; y Arcas, B.A. Communication-efficient learning of deep networks from decentralized data. In Proceedings of the 20th International Conference on Artificial Intelligence and Statistics, Fort Lauderdale, FL, USA, 20–22 April 2017; pp.1273–1282.
13. Yue, L.N.; Liu, Q.; Du, Y.C.; Gao, W.B.; Liu, Y.; Ya, F.Z. FedJudge: Federated Legal Large Language Model. *arXiv* **2024**, arXiv:2309.08173v3.
14. Xing, P.W.; Lu, S.T.; Yu, H. Federated Neuro-Symbolic Learning. *arXiv* **2024**, arXiv:2308.15324.
15. Jiang, J.G.; Liu, X.Y.; Fan, C.Y. Low-Parameter Federated Learning with Large Language Models. *arXiv* **2023**, arXiv:2307.13896.
16. OpenAI ChatGPT. Available online: https://openai.com/chatgpt (accessed on 5 August 2024).
17. Touvron, H.; Lavril, T.; Izacard, G.; Martinet, X.; Lachaux, M.A.; Lacroix, T.; Rozière, B.; Goyal, N.; Hambro, E.; Azhar, F.; et al. Llama: Open and efficient foundation language models. *arXiv* **2023**, arXiv:2302.13971.
18. Jiang, A.Q.; Sablayrolles, A.; Roux, A.; Mensch, A.; Savary, B.; Bamford, C.; Chaplot, D.S.; Casas, D.d.; Hanna, E.B.; Bressand, F.; et al. Mixtral of experts. *arXiv* **2024**, arXiv:2401.04088.
19. Anil, R.; Borgeaud, S.; Alayrac, J.; Yu, J.; Soricut, R.; Schalkwyk, J.; Dai, A.M.; Hauth, A.; Millican, K.; Silver, D.; et al. Gemini: A family of highly capable multimodal models. *arXiv* **2023**, arXiv:2312.11805.
20. Kaur,R.; Gabrijelcic,D.; Klobucar, T. Artificial intelligence for cybersecurity: Literature review and future research directions. *Inf. Fusion* **2023**, *97*, 101804. [CrossRef]
21. Kumar, S.; Gupta, U.; Singh, A.K.; Kishore Singh, A.K. Artificial intelligence: Revolutionizing cyber security in the digital era. *J. Comput. Mech. Manag.* **2023**, *2*, 31–42. [CrossRef]
22. Mijwil, M.; Aljanabi, M. Towards artificial intelligence-based cybersecurity: The practices and chatgpt generated ways to combat cybercrime. *Iraqi J. Comput. Sci. Math.* **2023**, *4*, 65–70.
23. da Silva, G.D.J.C.; Westphall, C.B. A survey of large language models in cybersecurity. *arXiv* **2024**, arXiv:2402.16968.
24. Motlagh, F.N.; Hajizadeh, M.; Majd, M.; Najafi, P.; Cheng, F.; Meinel, C. Large language models in cybersecurity: State-of-the-art. *arXiv* **2024**, arXiv:2402.00891.
25. Web API Dataset. Available online: https://github.com/gaozhirui233/openData (accessed on 18 October 2024).
26. Dettmers, T.; Pagnoni, A.; Holtzman, A.; Zettlemoyer, L. QLORA: Efficient finetuning of quantized LLMs. In Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS), New Orleans, LA, USA, 10–16 December 2023.
27. Chavan, A.; Liu, Z.; Gupta, D.; Xing, E.; Shen, Z. One-for-All: Generalized LoRA for parameter-efficient fine-tuning. *arXiv* **2023**, arXiv:2306.07967.
28. Zhang, Q.; Chen, M.; Bukharin, A.; Karampatziakis, N.; He, P.; Cheng, Y.; Chen, W.; Zhao, T. AdaLoRA: Adaptive budget allocation for parame-ter-efficient fine-tuning. *arXiv* **2023**, arXiv:2303.10512.
29. Hu, E.J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; Chen, W. Lora: Low-rank adaptation of large language models. *arXiv* **2021**, arXiv:2106.09685v2.
30. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention is all you need. In Proceedings of the 31st International Conference on Neural Information Processing Systems, Long Beach, CA, USA, 4–9 December 2017; Curran Associates, Inc.: Red Hook, NY, USA, 2017; pp. 6000–6010.
31. Gong, R.H.; Fan, Y.Q.; Wei, X.Y.; Bai, S.H.; Zhang, Y.C.; Zhang, X.G. Methodologies for the Implementation of Large Language Models in the Era of AI: Costs, Efficiency, and Effectiveness. *China Artif. Intell.* **2023**, *3*, 52–61. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.

Copyright of Applied Sciences (2076-3417) is the property of MDPI and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.